

GDG Siliguri — Code for Communities

Toy Train Edition · Project Brief for B1 + B6 + B7

Combined problem statement: Offline Landslide Reporter & Last-Mile Alert Relay (B1) + Road Status Mesh (B6) + Disaster-Ready Hills Assistant (B7), built as one unified civic tool per the official brief.

1. Event overview

A GDG Siliguri hackathon themed around the Darjeeling Himalayan Railway. Deliverables must be installable Progressive Web Apps that run fully offline, using on-device AI (Gemma models) for the intelligence layer — no server calls at demo time. Judged live with airplane mode on stage.

Submission requirements

- A public URL (GitHub Pages, Netlify, Vercel, or similar) that installs as a PWA and works after one online load.
- A public repo with a README stating: which on-device model(s) are used, the minimum device tested on, and what happens when the model is unavailable.
- A 2-minute demo video recorded with airplane mode visibly on.
- Teams of 1–2. One problem statement per team; combining statements is only pre-approved for B1+B6+B7 and A6+B6.

2. Judging criteria

Criterion	Weight	What is judged
Works offline	25%	Airplane mode on stage. Full flow completes. No silent server calls.
Usefulness to the hills	25%	Would a real passenger, farmer, host, or railway worker use this next monsoon?
On-device AI done well	20%	Model choice fits the device; graceful fallback; the AI adds something a form could not.
Craft	15%	Fast first load, small bundle, sensible caching, readable on a cheap phone in sunlight.
Belonging	15%	Nepali/Hindi/Bengali support; local names, places, and facts are right.

Note: every criterion above is execution-based, not tied to a specific track (A/B/C). Offline behavior, AI quality, craft, and language support are judged identically regardless of which problem statement is chosen. "Usefulness" explicitly names passenger, farmer, host, and railway worker as equally valid target users.

3. Source problem statements (verbatim scope)

B1 — Offline Landslide Reporter & Last-Mile Alert Relay (Medium)

In October 2025, over 300 mm of rain in 12 hours triggered landslides across the hills; an official high-risk warning had been issued hours earlier but never reached most people. Build both halves of that gap.

- **Report up:** a walker logs a slope observation — photo of a crack, seepage, a bulging retaining wall — and an on-device model classifies severity, explains the risk in plain language, and says what to do; reports queue in browser storage and sync when a hotspot appears.
- **Relay down:** the app can receive an official alert from one connected phone and rebroadcast it peer-to-peer to phones with no signal.
- **Bonus:** image analysis to flag tension cracks, tilted trees, fresh debris.

Suggested stack: getUserMedia + MediaPipe Tasks for Web + on-device LLM; IndexedDB + Background Sync for queue-and-sync; WebRTC / Web Bluetooth for the relay.

B6 — Road Status Mesh (Medium)

"Is NH-55 open? Is the Rohini road blocked?" is the daily monsoon question. Crowd-sourced blockage reports (photo, location, time, still-passable-or-not) that travel phone-to-phone over hotspot or Bluetooth when there is no network, with an on-device model de-duplicating and summarising them into a route status board.

Suggested stack: Geolocation + WebRTC data channels / Web Bluetooth + on-device LLM for summarisation; pairs naturally with A6 and B1.

B7 — Disaster-Ready Hills Assistant (Ambitious)

A broader offline companion for hill emergencies — landslide, cloudburst, road blockage, cold exposure, being cut off from the nearest hospital — giving step-by-step guidance and the nearest safe action when there is no signal to call for help. Teams choosing this should absorb B1 and B6 into one civic tool rather than build alongside them.

Suggested stack: On-device LLM + curated first-response knowledge base cached in the browser; Geolocation for nearest shelter/PHC.

4. Complete feature inventory (nothing scoped out)

B1 — Report-up

Feature	Detail
Photo capture	getUserMedia, in-app camera view
On-device classification	Crack, seepage, bulging retaining wall, plus bonus set: tension cracks, tilted trees, fresh debris
Risk explanation	On-device LLM turns the classification into a plain-language risk note
Recommended action	LLM also outputs what the walker should do next
Offline queue	IndexedDB storage, tagged queued / synced
Background sync	Auto-flips status to synced once connectivity returns

B1 — Relay-down

Feature	Detail
Alert ingestion	One connected phone receives an official alert
Peer rebroadcast	WebRTC data channel to nearby phones with no signal
Connection fallback	QR-code or manual SDP exchange if WebRTC signalling can't reach a server
Secondary transport	Web Bluetooth as a fallback path where WebRTC isn't viable

B6 — Road status mesh

Feature	Detail
Blockage reporting	Photo, location, time, passable / not-passable flag
Mesh propagation	Same phone-to-phone hotspot/Bluetooth transport as B1's relay — reused
Deduplication	On-device model clusters near-duplicate reports (same location/time window)
Summarization	On-device LLM turns the deduped cluster into one route-status line
Status board	Live view: road segment, current status, last updated

B7 — Disaster-ready assistant (unifying layer)

Feature	Detail
Curated knowledge base	Step-by-step guidance for landslide, cloudburst, road blockage, cold exposure, cut-off-from-hospital
Retrieval + Q&A;	On-device LLM answers grounded in the knowledge base (RAG pattern)
Nearest safe action	Geolocation + a static shelter/PHC list, distance-ranked
Unified shell	One app, not three bolted-together demos — required by B7's own brief

Cross-cutting requirements (apply everywhere, not once)

Criterion	What full coverage means here
Works offline	Every feature above functions with zero network calls; airplane mode never breaks any flow
On-device AI	Graceful fallback UI if the model fails to load, on every AI-touching screen
Craft	Fast cold load, lazy model loading, responsive layout, sunlight-readable high-contrast UI

Criterion	What full coverage means here
Belonging	Nepali / Hindi / Bengali / English on every screen and every LLM prompt and output
Usefulness	Distinct real value for a walker (B1), a driver (B6), and anyone stranded (B7)

5. Unified architecture

B1's relay and B6's mesh reuse the same transport layer for two data types (alerts vs. blockage reports). B1's risk explanation, B6's summarization, and B7's Q&A reuse the same on-device LLM wrapper for three prompt types. Build these two shared pieces once; the three feature modules become thin layers on top.

PWA app shell (offline-first, installs once, runs fully offline)

Shared core (used by every feature module below)

Module	Role
On-device LLM wrapper	One function: prompt in, text out. Every AI feature calls this, not a separate integration per feature.
Offline storage	IndexedDB + Background Sync. Single data layer for reports, alerts, and cached knowledge base.

Feature modules (built on the shared core)

Module	Covers
Report & note	B1 report-up: photo, on-device classification, LLM risk note, offline queue
Mesh & relay	B1 relay-down + B6: WebRTC/Bluetooth transport, alert rebroadcast, road-status dedup and summarization
Guide & routing	B7: emergency guidance Q&A; over the curated knowledge base, nearest shelter/PHC via geolocation

6. Build pipeline (0 to 100%)

Phase 0 — Environment & repo setup

Finish the gemma-model-setup guide (Ollama + Continue.dev) for the dev workflow. Init a git repo, pick a deploy target (GitHub Pages, Netlify, or Vercel), and scaffold a light project (Vite or plain HTML/JS/CSS — no heavy framework, keep the bundle small). Done when: a hello-world page is live at a public URL.

Phase 1 — PWA foundation

Add the web app manifest (name, icons, theme color), register a service worker, cache core assets (HTML/CSS/JS + model files) so the app installs and loads after one online visit. Build the tab shell: Report / Emergency Guide / Alerts. Done when: the shell loads correctly fully in airplane mode after one prior online visit.

Phase 2 — On-device AI core

Load a small Gemma variant via MediaPipe LLM Inference for Web (or Chrome's built-in Prompt API where supported). Write one reusable function: prompt in, text out. Add a graceful fallback for when the model fails to load. Done when: the function reliably returns a response from anywhere in the app, with a tested failure mode.

Phase 3 — B1 core vertical (report-up)

Wire getUserMedia for camera capture, run an on-device image classifier (MediaPipe Tasks for Web) to label severity, pass the label plus any user note into the LLM wrapper for a plain-language explanation and action, and store the report in IndexedDB. Done when: photo in, classification, LLM explanation, and storage all complete in airplane mode.

Phase 4 — Offline queue & sync

Add a queued/synced status field per report and a background check (Background Sync API, or a simple connectivity poll) that flips status once a connection appears. The 'server' side can be fully mocked. Done when: reports persist across reloads and visibly change status when network is toggled.

Phase 5 — B7 vertical: guidance + shelter routing

Write 10-15 emergency-guidance entries (landslide, cloudburst, road blockage, cold exposure, cut off from hospital); feed relevant entries into the Phase 2 LLM wrapper for a simple Q&A; screen. Add a static shelter/PHC list with a haversine-distance calculation against the user's Geolocation position. Done when: a guidance question gets a grounded answer, and the nearest shelter is shown with distance.

Phase 6 — Language, accessibility & craft pass

Add a language toggle (Nepali / Hindi / Bengali / English) covering both static UI strings and LLM prompts/outputs. Check contrast and font size for sunlight readability on a cheap phone; trim unused assets; lazy-load the model only when needed; re-measure cold load time. Done when: the app is fully usable end-to-end in each supported language, with a fast cold load on a throttled connection.

Phase 7 — Stretch, then final packaging

Only if time remains: attempt the B1 alert relay and/or B6 road-status mesh via WebRTC/Bluetooth, time-boxed hard. Then freeze scope, run a full airplane-mode pass on every feature, write the README (model(s) used, minimum device tested, fallback behavior), record the 2-minute demo video with airplane mode visibly on, and do a final deploy. Done when: a stranger could open the repo, read the README, and understand exactly what was

built and its limits — and the video shows the full flow working offline.

Cut-line if time runs short

If time runs out, ship B1 (report-up + queue) + B7 (guidance + shelter routing) as a working, polished civic tool, and drop B6 and the relay entirely. A working two-thirds beats a broken whole — this still satisfies the combo brief as a legitimate, demoable submission.